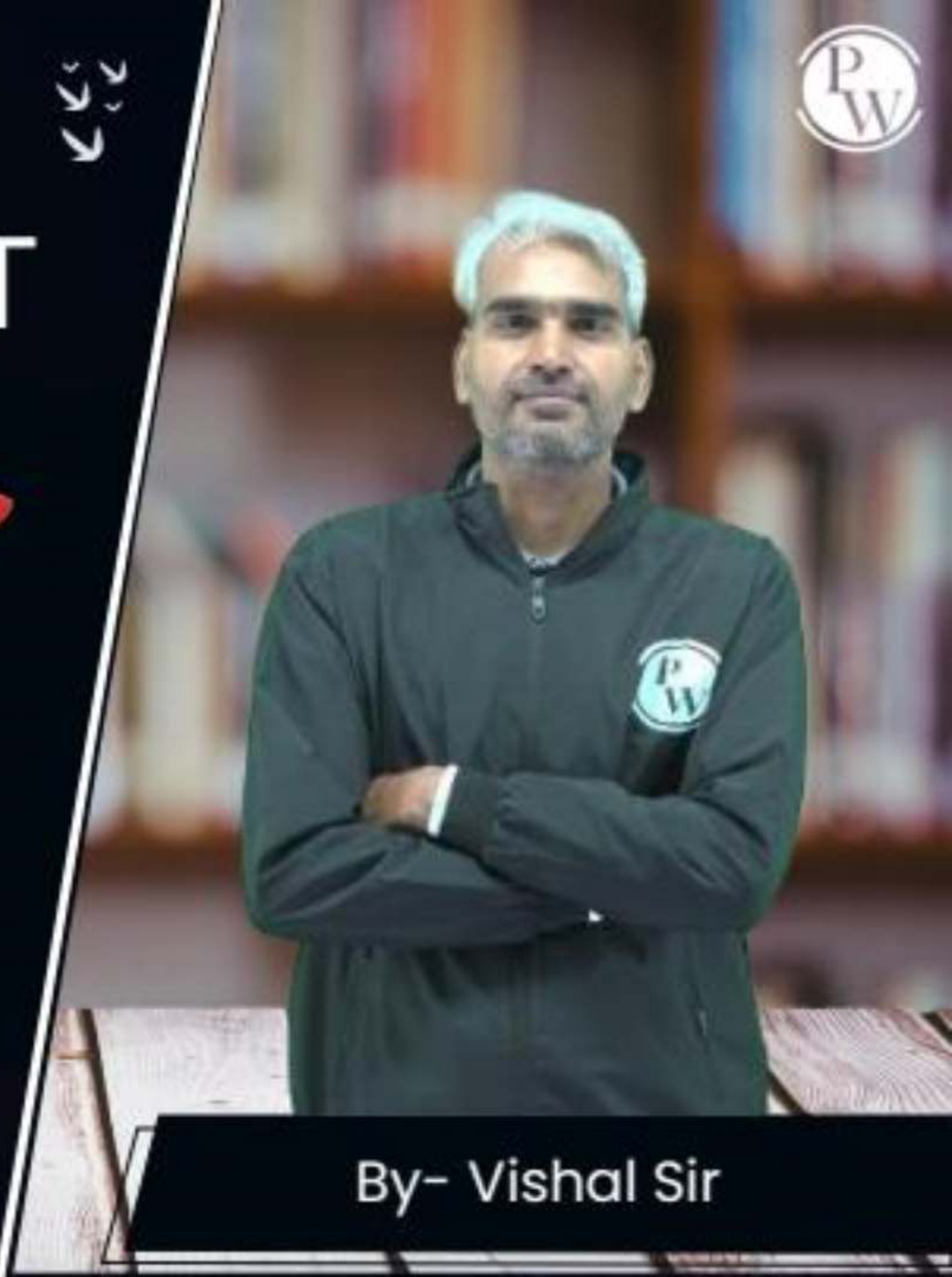
Computer Science & IT



Database Management System

Transaction & concurrency control

Lecture No. 05



By- Vishal Sir

Recap of Previous Lecture



- Topic: Problems because of concurrent execution
- Topic: RW problem
- Topic: WR problem
- Topic: WW problem
- Topic: Lost update problem

Slide

Topics to be Covered



- Topic: Classification of schedule
- Topic: Irrecoverable schedule
- Topic: Recoverable schedule
- Topic: Cascading rollback problem
- Topic: Cascadeless schedule
- Topic: Strict recoverable schedule

Classification based on Recoverability

Slide



Topic : Classification of schedule

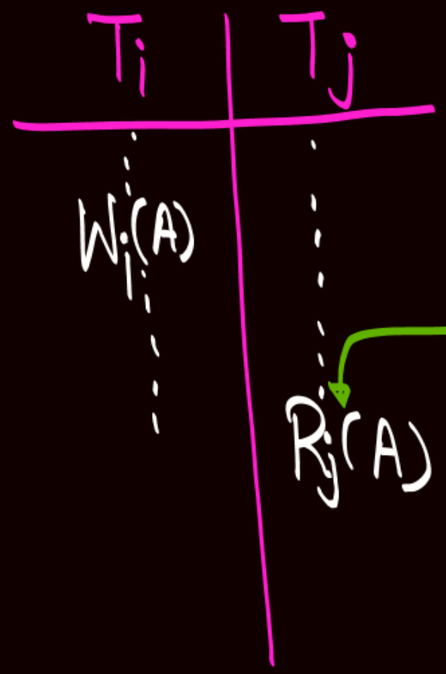
Classification based on Recoverability

1. Irrecoverable schedule
2. Recoverable schedule
3. Cascadeless rollback recoverable schedule
4. Strict recoverable schedule

Classification based on Serializability

1. Conflict serializable schedule
2. View serializable schedule

Note :- If transaction T_j reads the value written by an uncommitted transaction T_i, then transaction T_j is dependent on transaction T_i.



T_j reads the value of dataitem 'A' written by T_i
∴ T_j is dependent on T_i

If transaction T_i rollback for some reason then we also need to rollback transactions dependent on transaction T_i {i.e, T_j in our eg.}

We can rollback a transaction only if that transaction has not executed its Commit operation

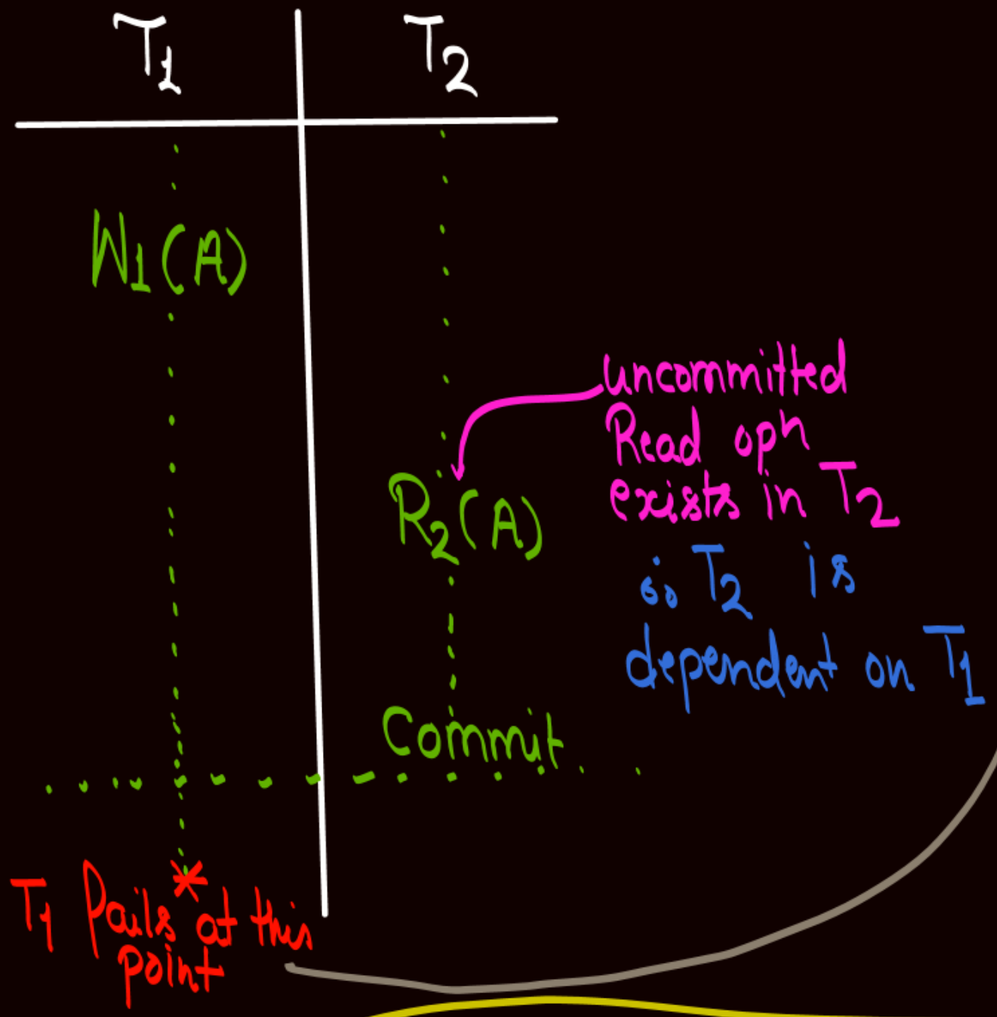
Rollback of a committed transaction is not allowed.



Topic : Irrecoverable schedule

- It is not possible to rollback a committed transaction.
- If there arise a situation when we are required to rollback a committed transaction, then it will not be possible to rollback that transaction. And hence we will not be able to recover from that failure, such schedules are called irrecoverable schedule

Irrecoverable Schedule



T_1 failed, ∴ We need to rollback transaction T_1.

Because of rollback of T_1 We need to rollback all the transactions that are dependent on T_1 { in our example, we need to rollback T_2 }

But T_2 has committed

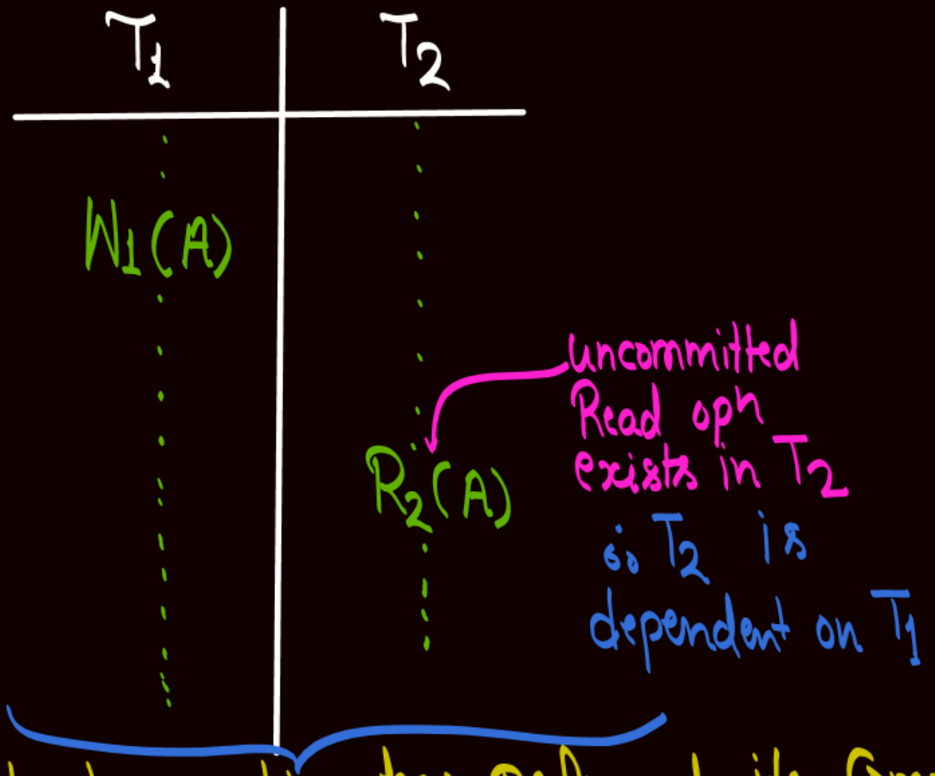
∴ We can not rollback T_2

Hence we can not recover from this failure.

Therefore

Given schedule is an irrecoverable schedule

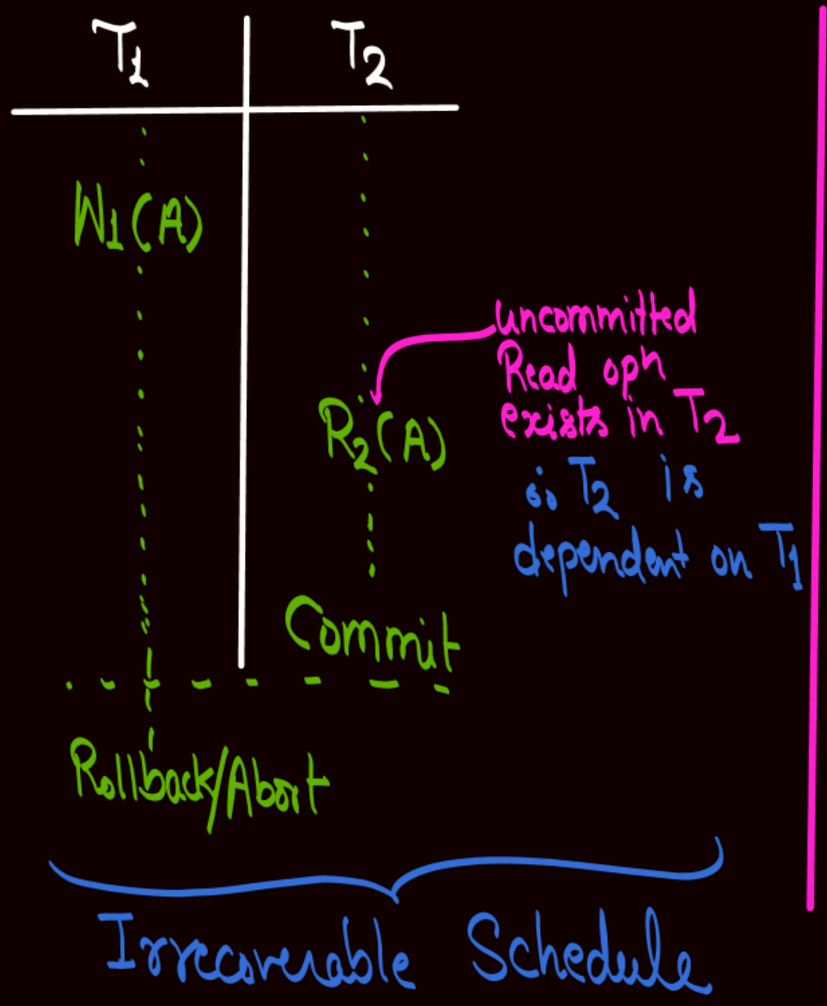
Irrecoverable Schedule



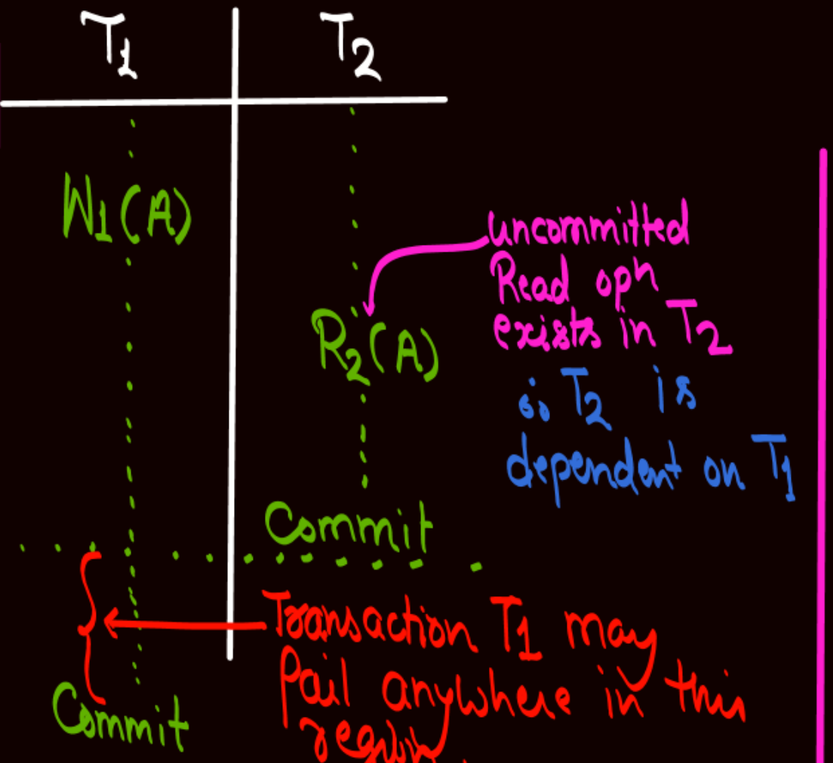
- No transaction has performed its Commit oph
  ie If any failure occur, then we can rollback both transactions
  i.e, we will be able to recover from failure

Hence, Schedule is recoverable schedule

Irrecoverable Schedule

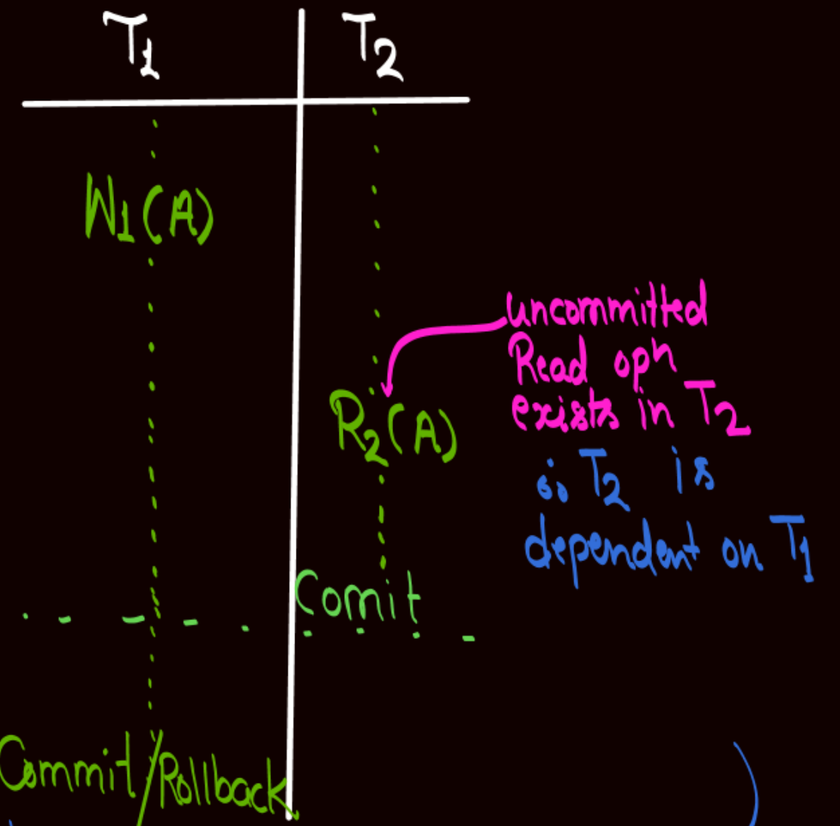


Irrecoverable Schedule



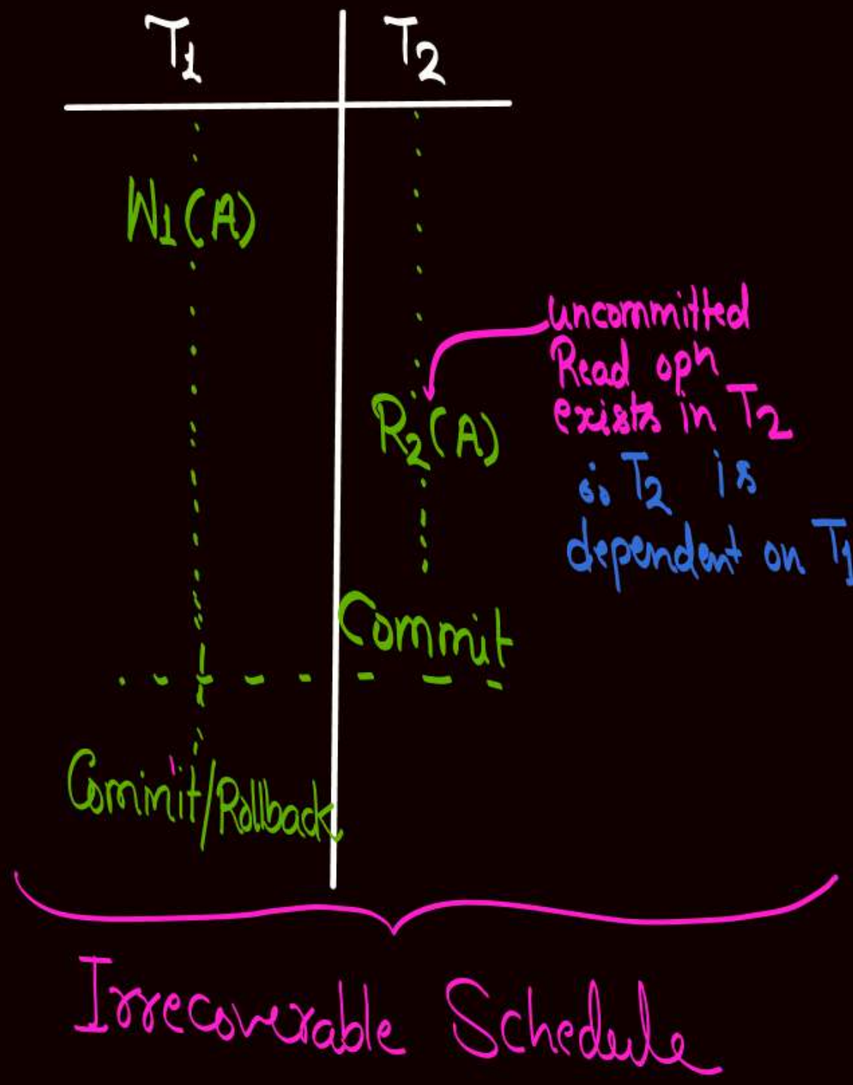
After commit of transaction T2 schedule is unsafe because T1 may fail after commit of T2 and before commit of T1, if this happens then we will not be able to recover from failure.

Hence it is also an irrecoverable schedule



Irrecoverable Schedule

Irrecoverable Schedule



* Let two transactions T_1 & T_2 exists in a schedule such that T_2 is dependent on T_1,

  If transaction T_2 {i.e. dependent transaction} Commit before the commit/Rollback of transaction T_1 {i.e, transaction on which T_2 depends}, then such schedule is called "Irrecoverable Schedule"

Irrecoverable Schedule

Uncommitted read oph are necessary (but not sufficient) for a schedule to be irrecoverable Schedule.

i.e, ① If uncommitted read oph does not exist in a schedule then that schedule is always recoverable

and ② If uncommitted read oph exists in a schedule then that schedule may or may not be an irrecoverable schedule.

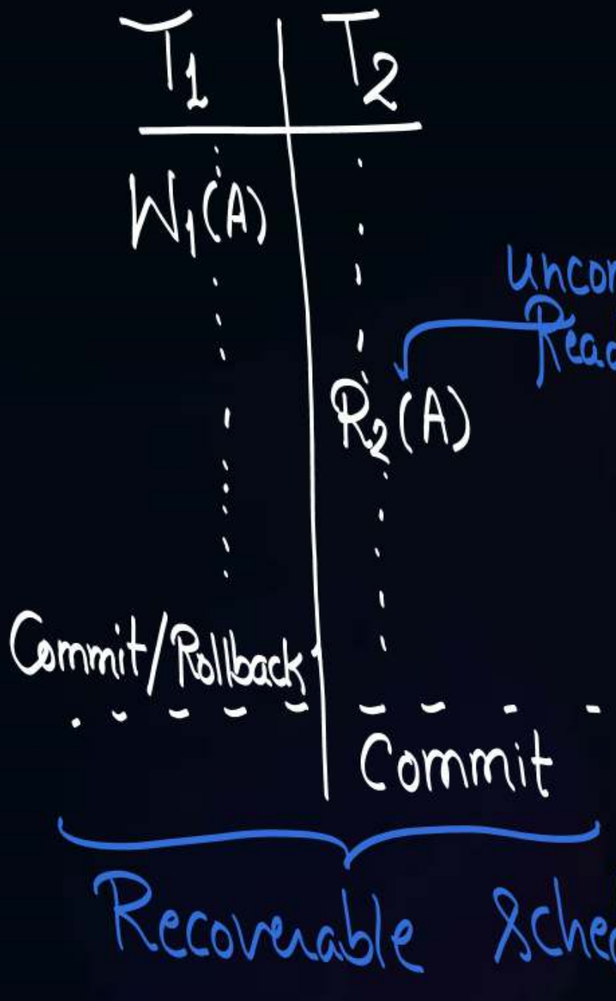


Topic : Recoverable schedule

For a schedule to be called recoverable schedule,

either, uncommitted read oph must not exist in the schedule

or, if uncommitted read oph exists in the schedule, then Commit oph of dependent transaction must appear after the Commit/Rollback of the transaction on which it depends.



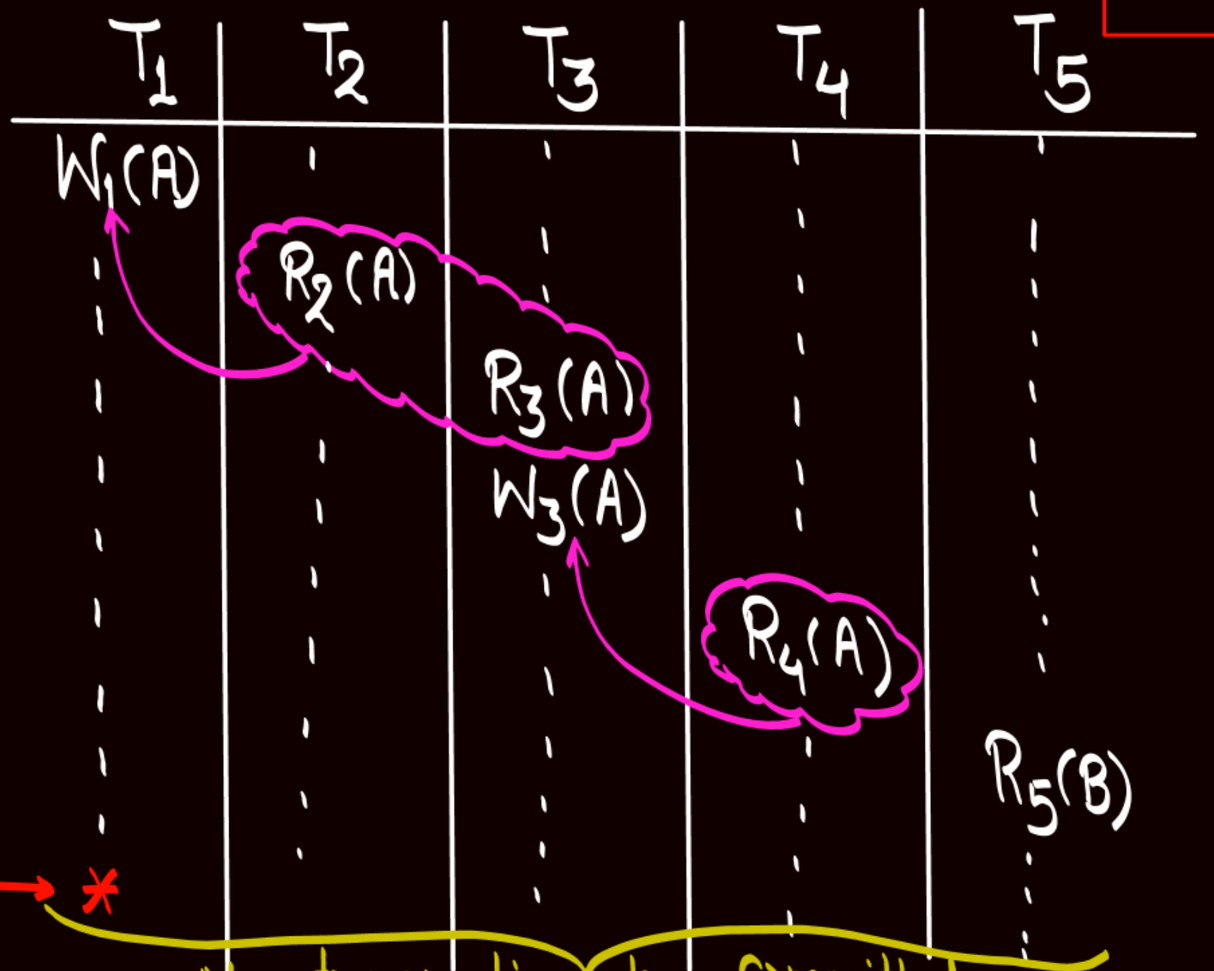


Topic : Cascading rollback problem

If failure of one transaction results in rollback of a set of other transactions along with failed transaction, then it is called Cascading rollback.

And we say that the Schedule suffer from Cascading rollback problem.

Cascading rollback problem



Cascading rollback increases Wastage of CPU time & IO time

* If T_1 fails, we rollback T_1.
* Because of rollback of T_1 we need to rollback transactions dependent on T_1, i.e, T_2 & T_3
* No other transaction depends on T_2 but T_4 depends on T_3
∴ Because of rollback of T_3 we will rollback T_4 as well.
* No other transaction depends on T_4

Because of failure of T_1, we need to rollback T_2, T_3 & T_4 as well along with T_1 ∴ Cascading rollback Problem exist.

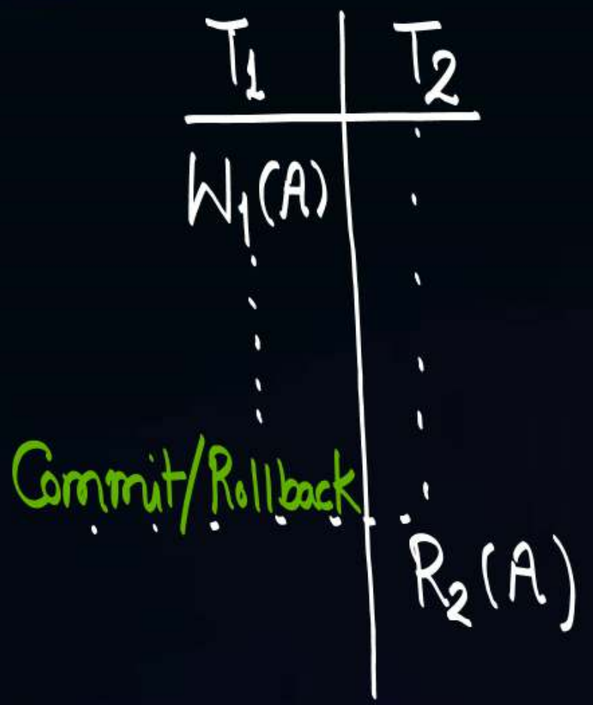


Topic : Cascadeless recoverable schedule

- If uncommitted read opn exist in the schedule then dependency will exist among the transactions, and if dependency exists among the transaction then because of failure of one transaction we may have to rollback some other transactions as well, i.e., Cascading rollback Problem will exist.

- For Cascadeless recoverable schedule, uncommitted read operation must not be allowed

Topic : Cascadeless recoverable schedule

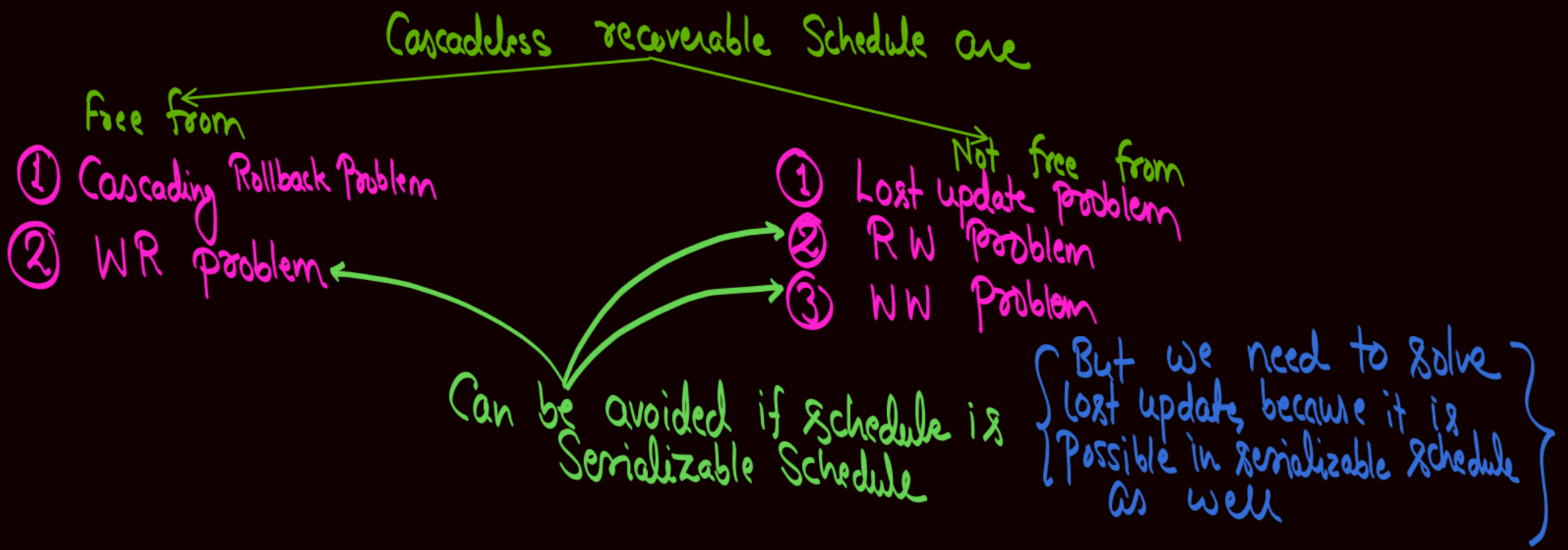


- For cascadeless recoverable schedule uncommitted read op^n must not be allowed.
- For cascadeless recoverable schedule,
  If transaction T_1 updates the dataitem(A), then no other transaction should be allowed to read the value of dataitem 'A' written by T_1 until the commit/Rollback of T_1.

Note:- Uncommitted read oph are possible in Serializable as well as non-Serializable Schedule.

i.e., Cascading rollback problem is possible in Serializable Schedule as well.

Hence we had to find the solution for Cascading rollback problem.





Topic : Strict recoverable schedule

Strict recoverable schedule is Cascadeless recoverable schedule as well as it is free from lost-update problem

(Cascadeless Recoverable Schedule (i.e, No uncommitted Read) + No lost-update Problem (i.e, No simultaneous write-write opn)) = Strict Recoverable Schedule

T_1	T_2
$W_1(A)$	
Commit/Rollback	$R_2(A)$

+

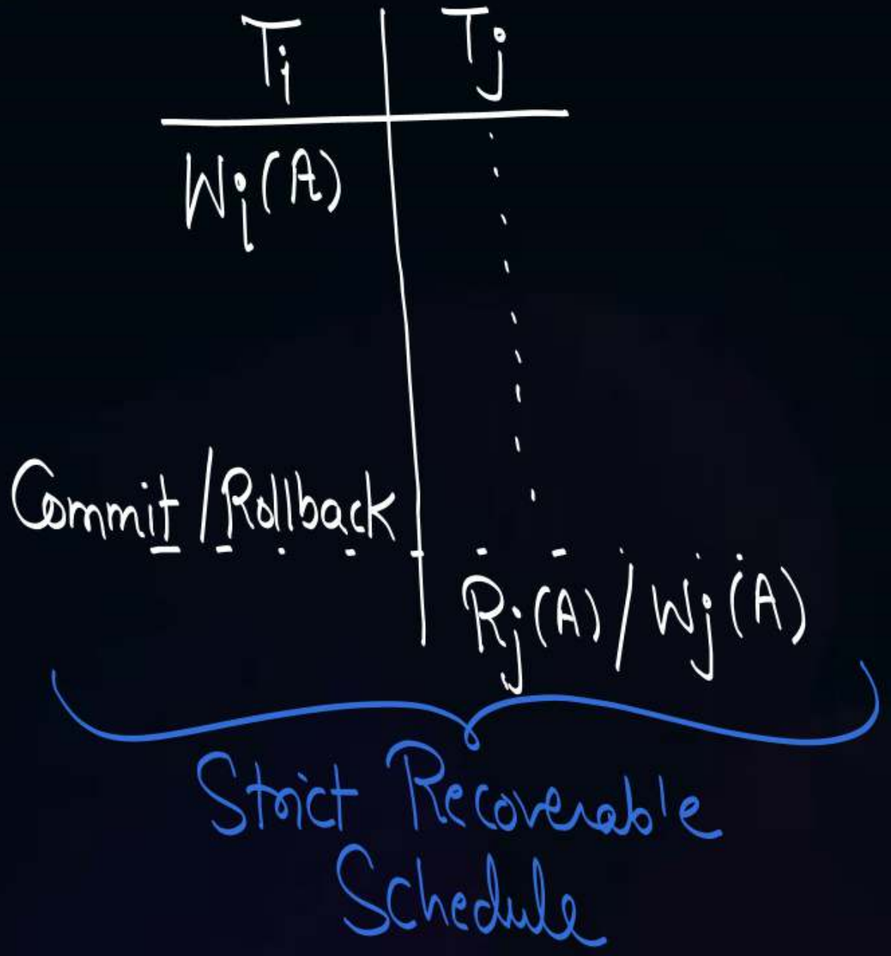
T_1	T_2
$W_1(A)$	
Commit/Rollback	$W_2(A)$

=

T_1	T_2
$W_1(A)$	
Commit/Rollback	$R_2(A)/W_2(A)$



Topic : Strict recoverable schedule



For a schedule to be called strict recoverable, if transaction Ti updates any dataitem 'A', then no other transaction Tj is allowed to read or write the dataitem 'A' until the commit or Rollback of transaction Ti

Strict Recoverable Schedule

Ex.

Free from

1. Cascading Rollback Problem
2. Lost-update problem
3. WR problem
4. WW Problem

Not free from

1. RW problem

Can be solved by Ensuring "Serializability"

Criteria for Consistency :-

1. Schedule must be Strict recoverable
2. Schedule must be Serializable

These two can solve all problems



2 mins Summary

- **Topic** Classification of schedule
- **Topic** Irrecoverable schedule
- **Topic** Recoverable schedule
- **Topic** Cascading rollback problem
- **Topic** Cascadeless schedule
- **Topic** Strict recoverable schedule

THANK - YOU